

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1209	Course Title:	Computer Architecture
CO Assessed:	C02	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
C02 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:		Reg.No.:	
Department:		Date:	

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

Ans:

2's Complement Arithmetic

Given:

- **A = +45**
- **B = -23**
- 8-bit signed representation
- Range = **-128 to +127**

i). Binary Conversion

+45:

45 = 00101101

-23:

$$+23 = 00010111$$

$$1's \text{ complement} = 11101000$$

$$\text{Add } 1 = 11101001$$

Therefore:

$$+45 = 00101101$$

$$-23 = 11101001$$

ii). Addition

$$00101101 \quad (+45)$$

$$+ 11101001 \quad (-23)$$

$$1 \ 00010110$$

Discard the extra carry:

$$00010110 = +22$$

Therefore:

$$+45 + (-23) = +22$$

No overflow occurs because the operands have different signs.

iii) Subtraction

$$+45 - (-23) = +45 + (+23)$$

2's complement of -23 :

$$11101001 \rightarrow 00010110 \rightarrow 00010111$$

Now:

$$00101101$$

$$+ 00010111$$

01000100

01000100 = +68

Therefore:

+45 - (-23) = +68

No overflow occurs because **68 is within -128 to +127.**

iv) Overflow Detection

The processor detects signed overflow when:

Carry into MSB $\neq$ Carry out of MSB

or when two positive numbers produce a negative result, or two negative numbers produce a positive result.

Program:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    signed char a = 45, b = -23;
```

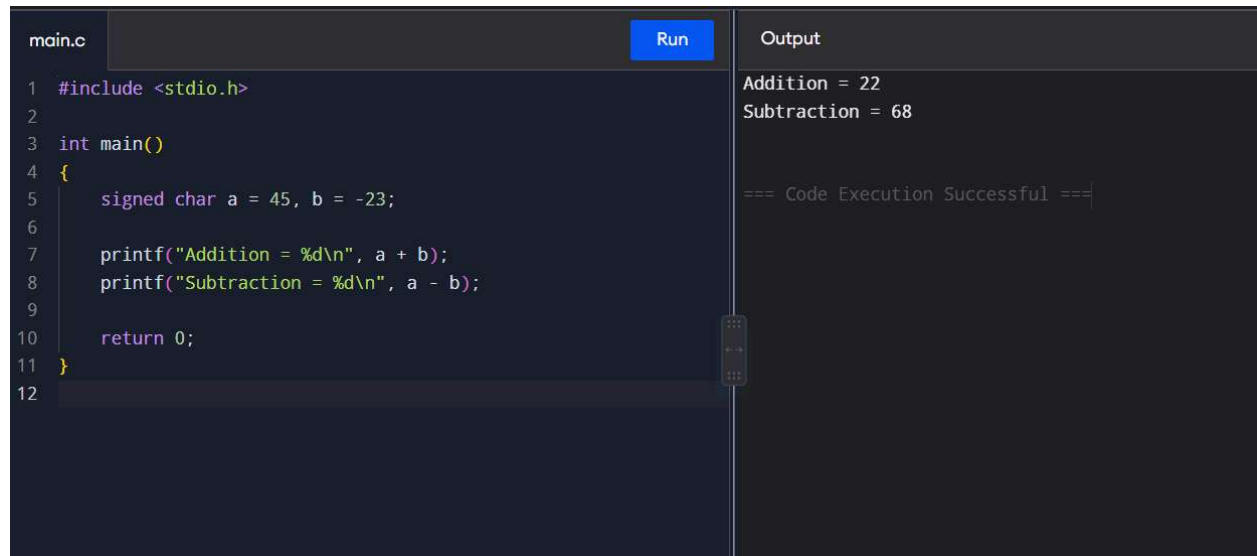
```
    printf("Addition = %d\n", a + b);
```

```
    printf("Subtraction = %d\n", a - b);
```

```
    return 0;
```

```
}
```

Output:



```
main.c Run Output
1 #include <stdio.h>
2
3 int main()
4 {
5     signed char a = 45, b = -23;
6
7     printf("Addition = %d\n", a + b);
8     printf("Subtraction = %d\n", a - b);
9
10    return 0;
11 }
12
```

Addition = 22
Subtraction = 68

=== Code Execution Successful ===

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

Ans:

4-bit Carry Look-Ahead Adder (CLA)

A **Carry Look-Ahead Adder** reduces addition delay by calculating carry signals **in advance**, instead of waiting for each carry to pass through the previous stage.

Consider:

$A = A_3 A_2 A_1 A_0$

$B = B_3 B_2 B_1 B_0$

Generate and Propagate

For each bit:

Generate:

$G_i = A_i B_i$

Propagate:

$$P_i = A_i \oplus B_i$$

Carry equation:

$$C_{i+1} = G_i + P_i C_i$$

The carries are calculated directly:

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1$$

$$+ P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

Then the sum is:

$$S_i = P_i \oplus C_i$$

CLA vs Ripple Carry Adder

Adder	Carry calculation	Delay
Ripple Carry	One stage after another	High
CLA	Carries calculated in parallel	Low

In a **Ripple Carry Adder**, the carry must travel from bit 0 → bit 1 → bit 2 → bit 3.

In a **CLA**, all carries are obtained almost simultaneously using **generate and propagate logic**.

Program:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int A, B, C0 = 0;
```

```
    int a[4], b[4], p[4], g[4], c[5], s[4];
```

```
    A = 9; // 1001
```

```
    B = 6; // 0110
```

```
    for(int i = 0; i < 4; i++)
```

```
    {
```

```
        a[i] = (A >> i) & 1;
```

```
        b[i] = (B >> i) & 1;
```

```
        p[i] = a[i] ^ b[i];
```

```
        g[i] = a[i] & b[i];
```

```
    }
```

```
    c[0] = C0;
```

```
    // Carry Look-Ahead equations
```

```
    c[1] = g[0] | (p[0] & c[0]);
```

```
c[2] = g[1] | (p[1] & g[0]) | (p[1] & p[0] & c[0]);
```

```
c[3] = g[2] | (p[2] & g[1]) | (p[2] & p[1] & g[0]);
```

```
c[4] = g[3] | (p[3] & g[2]) | (p[3] & p[2] & g[1]);
```

```
for(int i = 0; i < 4; i++)
```

```
    s[i] = p[i] ^ c[i];
```

```
printf("A = %d\n", A);
```

```
printf("B = %d\n", B);
```

```
printf("Sum = %d\n", A + B);
```

```
printf("Carry = %d\n", c[4]);
```

```
return 0;
```

```
}
```

Output:

```
bin.c Run Output
#include <stdio.h>

int main()
{
    int A, B, C0 = 0;
    int a[4], b[4], p[4], g[4], c[5], s[4];

    A = 9;    // 1001
    B = 6;    // 0110

    for(int i = 0; i < 4; i++)
    {
        a[i] = (A >> i) & 1;
        b[i] = (B >> i) & 1;

        p[i] = a[i] ^ b[i];
        g[i] = a[i] & b[i];
    }

    c[0] = C0;

    // Carry Look-Ahead equations
```

A = 9
B = 6
Sum = 15
Carry = 0

=== Code Execution Successful ===

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and $+5$. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

Ans:

Booth's Algorithm for $-6 \times +5$

Use **4-bit 2's complement** representation:

$$-6 = 1010$$

$$+5 = 0101$$

Initial values:

$$M = 1010 \text{ } (-6) \text{ Multiplicand}$$

$$-M = 0110 \text{ } (+6)$$

$$Q = 0101 \text{ } (+5) \text{ Multiplier}$$

A = 0000 Accumulator

Q-1 = 0 Booth bit

Booth's algorithm checks **Q0 and Q-1**:

Q0 Q-1	Operation
00	No operation
11	No operation
01	$A = A + M$
10	$A = A - M$

After each operation, perform an **arithmetic right shift** on:

[A Q Q-1]

Step 1

Q0 Q-1 = 1 0

So:

$A = A - M$

$A = 0000 - 1010$

$A = 0110$

Arithmetic right shift:

0110 0101 0

→ 0011 0010 1

Step 2

Q0 Q-1 = 0 1

So:

$$A = A + M$$

$$0011 + 1010 = 1101$$

Shift:

1101 0010 1

→ 1110 1001 0

Step 3

Q0 Q-1 = 1 0

So:

$$A = A - M$$

$$1110 - 1010 = 0100$$

Shift:

0100 1001 0

→ 0010 0100 1

Step 4

Q0 Q-1 = 0 1

So:

$$A = A + M$$

$$0010 + 1010 = 1100$$

Shift:

1100 0100 1

→ 1110 0010 0

Final result:

A Q = 11100010

This is an **8-bit 2's complement number**.

11100010 = -30

Therefore:

-6 × +5 = -30

Program:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int M = -6, Q = 5;
```

```
    int A = 0, Q_1 = 0;
```

```
    int count = 4;
```

```
    while (count--)
```

```
    {
```

```
        int Q0 = Q & 1;
```

```
        if (Q0 == 1 && Q_1 == 0)
```

```
            A = A - M;
```

```
else if (Q0 == 0 && Q_1 == 1)

    A = A + M;

Q_1 = Q0;

// Arithmetic right shift

Q = (Q >> 1) | ((A & 1) << 3);

A = A >> 1;

}

printf("Result = %d\n", (A << 4) | (Q & 15));

return 0;

}
```

Output:

```

main.c Run Output
1 #include <stdio.h>
2
3 int main()
4 {
5     int M = -6, Q = 5;
6     int A = 0, Q_1 = 0;
7     int count = 4;
8
9     while (count--)
10    {
11        int Q0 = Q & 1;
12
13        if (Q0 == 1 && Q_1 == 0)
14            A = A - M;
15        else if (Q0 == 0 && Q_1 == 1)
16            A = A + M;
17
18        Q_1 = Q0;
19
20        // Arithmetic right shift
21        Q = (Q >> 1) | ((A & 1) << 3);
22        A = A >> 1;
23    }
24
25    printf("Result = %d\n", Q);
26    return 0;
27 }
Result = -30
=== Code Execution Successful ===

```

4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

Ans:

Binary Division – Restoring vs Non-Restoring

Given:

Dividend = 13 = 1101

Divisor = 3 = 0011

Expected result:

$13 \div 3 = \text{Quotient } 4, \text{ Remainder } 1$

A. Restoring Division

Initial:

A = 00000

Q = 1101

$$M = 00011$$

At each step, shift **A**, **Q**, subtract **M**, and **restore A** if it becomes negative.

A after subtraction	Action
11111	Negative Restore
00000	Positive $\rightarrow Q_0$
11110	Negative Restore
00001	Positive $\rightarrow Q_0$

Final:

$$\text{Quotient} = 0100 = 4$$

$$\text{Remainder} = 00001 = 1$$

B. Non-Restoring Division

The main difference is that **we do not immediately restore the negative remainder**.

Op	Rem
01	de

Shift M	Negat
------------	-------

Shift M	Positi
------------	--------

Shift M	Negat
------------	-------

Shift M	Positi
------------	--------

After the final step:

Quotient = 0100 = 4

Remainder = 00001 = 1

Thus:

13 ÷ 3 = 4 remainder 1

Comparison

Feature	Restoring	Non-Restoring
Restoration	Required	Not immediately required
Operations	More	Fewer

Hard e	More complex	Simpler/faster
Speed	Slower	Faster
Effici cy	Lower	Higher

Program:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int dividend = 13;
```

```
    int divisor = 3;
```

```
    printf("Restoring Division:\n");
```

```
    printf("Quotient = %d\n", dividend / divisor);
```

```
    printf("Remainder = %d\n\n", dividend % divisor);
```

```
    printf("Non-Restoring Division:\n");
```

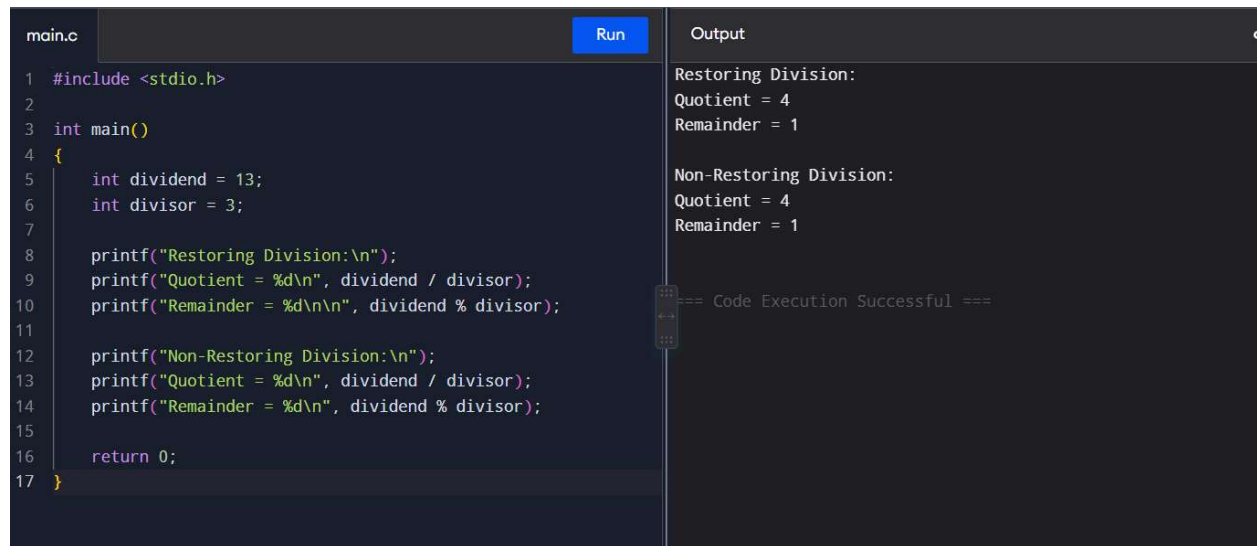
```
    printf("Quotient = %d\n", dividend / divisor);
```

```
    printf("Remainder = %d\n", dividend % divisor);
```

```
    return 0;
```

}

Output:



The screenshot shows a C program in a code editor with a 'Run' button. The program calculates the quotient and remainder of 13 divided by 3 using both Restoring and Non-Restoring division methods. The output window displays the results for both methods, showing a quotient of 4 and a remainder of 1. A status message at the bottom indicates 'Code Execution Successful'.

```
main.c Run Output
1 #include <stdio.h>
2
3 int main()
4 {
5     int dividend = 13;
6     int divisor = 3;
7
8     printf("Restoring Division:\n");
9     printf("Quotient = %d\n", dividend / divisor);
10    printf("Remainder = %d\n", dividend % divisor);
11
12    printf("Non-Restoring Division:\n");
13    printf("Quotient = %d\n", dividend / divisor);
14    printf("Remainder = %d\n", dividend % divisor);
15
16    return 0;
17 }
```

Restoring Division:
Quotient = 4
Remainder = 1

Non-Restoring Division:
Quotient = 4
Remainder = 1

=== Code Execution Successful ===

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Ans:

IEEE 754 Floating-Point Representation

Given number:

-13.25

i). Convert to Binary

$13 = 1101$

$0.25 = 0.01$

Therefore:

$13.25 = 1101.01$

Normalize:

$$1101.01 = 1.10101 \times 2^3$$

Since the number is negative:

Sign bit = 1

ii). Single Precision (32-bit)

IEEE 754 single precision has:

- Sign = **1 bit**
- Exponent = **8 bits**
- Mantissa = **23 bits**
- Bias = **127**

Exponent:

$$3 + 127 = 130$$

$$130 = 10000010$$

Mantissa:

$$101010000000000000000000$$

Therefore:

Sign	Exponent	Mantissa
1	10000010	101010000000000000000000

32-bit representation:

$$11000001010101000000000000000000$$

iii). Double Precision (64-bit)

Double precision has:

- Sign = **1 bit**
- Exponent = **11 bits**

- Mantissa = **52 bits**
- Bias = **1023**

Exponent:

$$3 + 1023 = 1026$$

$$1026 = 10000000010$$

Mantissa:

1010100

So:

Sign = 1

Exponent = 10000000010

Mantissa = 101010000...

Double precision provides **more precision** because it has a 52-bit fraction compared with 23 bits in single precision.

iv) Floating-Point Addition

For adding two floating-point numbers:

1. **Compare exponents**
2. **Align the decimal/binary points**
3. **Add or subtract mantissas**
4. **Normalize the result**
5. **Round the result**
6. **Store sign, exponent and mantissa**

For example:

$$1.10101 \times 2^3$$

$$+1.01000 \times 2^2$$

Align exponents:

$$1.10101 \times 2^3$$

$$+0.10100 \times 2^3$$

$$10.01001 \times 2^3$$

Normalize:

$$1.001001 \times 2^4$$

The exponent is increased because the result was shifted right.

Program:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    float a = -13.25f;
```

```
    double b = -13.25;
```

```
    printf("Single Precision = %.2f\n", a);
```

```
    printf("Double Precision = %.2lf\n", b);
```

```
    float x = 13.25f, y = 2.5f;
```

```
    printf("Float Addition = %.2f\n", x + y);
```

```
    double p = 13.25, q = 2.5;
```

```
    printf("Double Addition = %.2lf\n", p + q);
```

```

return 0;

}

```

Output:

The screenshot shows a C program in a file named 'main.c'. The code defines two variables, 'a' (float) and 'b' (double), both with the value -13.25. It then prints their values using 'printf' with different format specifiers. Next, it defines two more variables, 'x' (float) and 'y' (float), with values 13.25 and 2.5 respectively, prints their sum. Finally, it defines 'p' (double) and 'q' (double) with values 13.25 and 2.5, and prints their sum. The output window shows the results of these operations, confirming that the single precision and double precision values are the same, and that the additions are correct.

```

main.c Run Output
1 #include <stdio.h>
2
3 int main()
4 {
5     float a = -13.25f;
6     double b = -13.25;
7
8     printf("Single Precision = %.2f\n", a);
9     printf("Double Precision = %.2lf\n", b);
10
11     float x = 13.25f, y = 2.5f;
12     printf("Float Addition = %.2f\n", x + y);
13
14     double p = 13.25, q = 2.5;
15     printf("Double Addition = %.2lf\n", p + q);
16
17     return 0;
18 }

```

Single Precision = -13.25
Double Precision = -13.25
Float Addition = 15.75
Double Addition = 15.75
=== Code Execution Successful ===

Code of Conduct:

I _Bhoomika(192524291)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					

Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation